DevHub System Design

1. Introduction

The Smart Pantry Manager is a native Android app (Java) created for the Mobile Application Development 700 assignment. It tackles a common issue: you buy food, store it in the pantry, then forget about it until it spoils. When you finally ask, "what can I cook tonight?", the answer often ends up being takeaway. This app keeps a digital record of your pantry and helps by suggesting recipes you can make with the ingredients you already have.

The app was built with the Android SDK in Java and uses the Model-View-ViewModel (MVVM) architecture, as recommended by Google. It stores data with SQLite through the Room persistence library, and the UI updates automatically using LiveData. The date picker, text fields, and bottom navigation all use Material Design components (Material 3), giving the app a modern Android look.

The functional scope of the application covers:

- Pantry management: the ability to add, edit and delete pantry items (with a name, quantity, unit (g, kg, ml, tbsp, tsp, cups or pieces) and an optional expiry date);
- Strict recipe matching: the Suggested screen will only list recipes where every ingredient required is present in the pantry in at least the required amount.
- A bonus 'Almost There' list: recipes missing just one ingredient are shown separately so you know what to pick up next;
- Recipe detail view: ingredients with required quantities and numbered preparation steps.

2. System Design

2.1 Architecture

The app is built on the MVVM pattern. The fragments (Pantry, Suggested, Settings, Add/Edit, Recipe Detail) all observe LiveData exposed by the ViewModels. The ViewModels in turn delegate to Repositories which call the Room DAOs. As every query returns LiveData the UI will update automatically as the underlying SQLite data changes. No manual refresh is required after an insert, update or delete.

The matching engine is a utility class called IngredientMatcher. Before comparing a recipe to the pantry, it normalizes both sides. Ingredient names are trimmed, changed to lowercase, and made singular (for example, "tomatoes" becomes "tomato"). Quantities are converted to base units like grams, millilitres, or pieces, so "200 g butter" can be matched with "0.25 kg butter" in the pantry. A recipe only matches if every required ingredient is present in the pantry, in the correct unit, and in enough quantity.

2.2 Screen Flow

The app has three main screens you can reach from the bottom navigation: Pantry (which opens first), Suggested, and Settings. On the Pantry screen, you can open the Add/Edit form, which includes a Material date picker for the expiry date. If you long-press a pantry item, a delete confirmation dialog appears with an option to undo. On the Suggested screen, tapping a recipe opens its detail view.

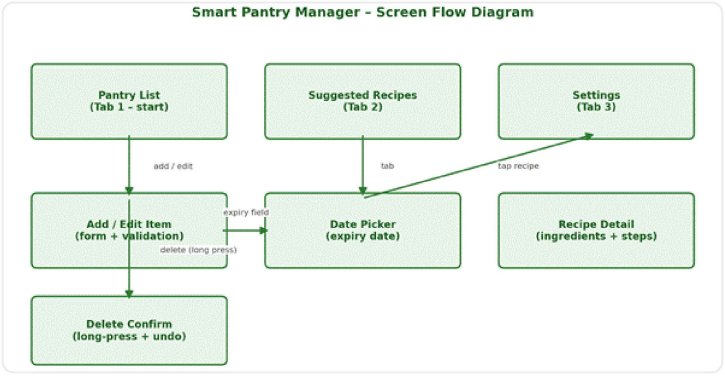


Figure 1: Screen flow diagram of the Smart Pantry Manager.

2.3 Data Model (ER Diagram)

The Room database has four entities. recipes is the parent of both recipe_ingredients and recipe_steps via one-to-many foreign key relationships with ON DELETE CASCADE. pantry_items is a standalone entity (intentionally not linked by a foreign key) as recipes are matched to stock at query time by comparing normalised ingredient names rather than by fixed IDs. This makes the matching process tolerant of things like plurals.

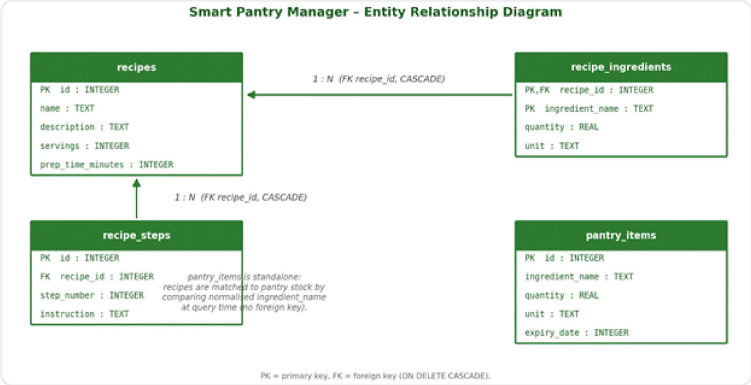


Figure 2: Entity-relationship diagram of the Room database.

3. Screenshots of Application Output

All the screenshots below were taken on a real Android device using the final version of the app.

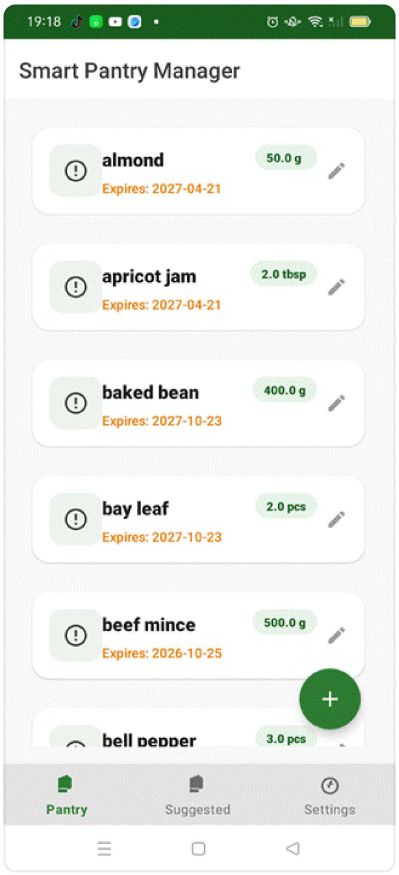


Figure 3: Pantry list with items, quantity badges and edit actions.

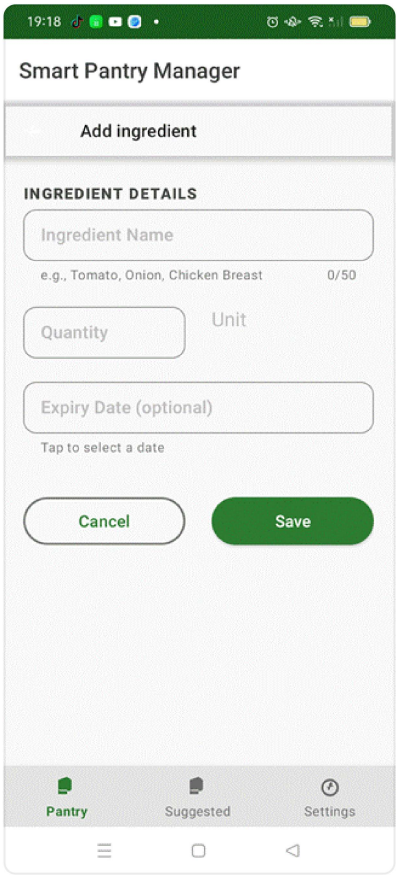


Figure 4: Add Ingredient form with name, quantity, unit and expiry date fields.

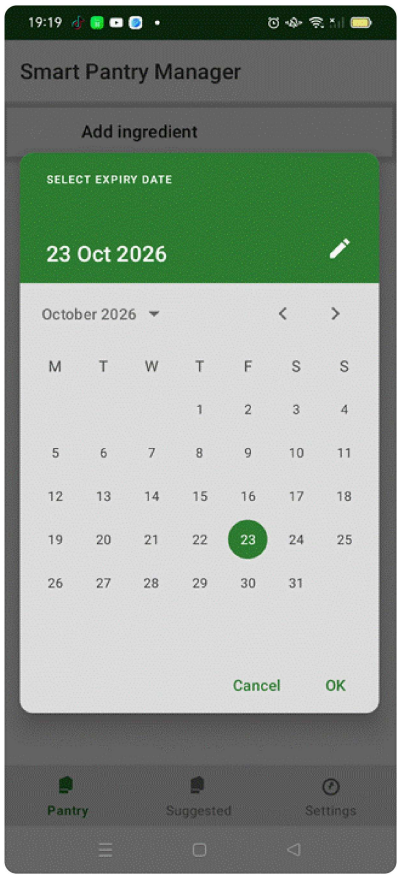


Figure 5: Material date picker for selecting the expiry date.

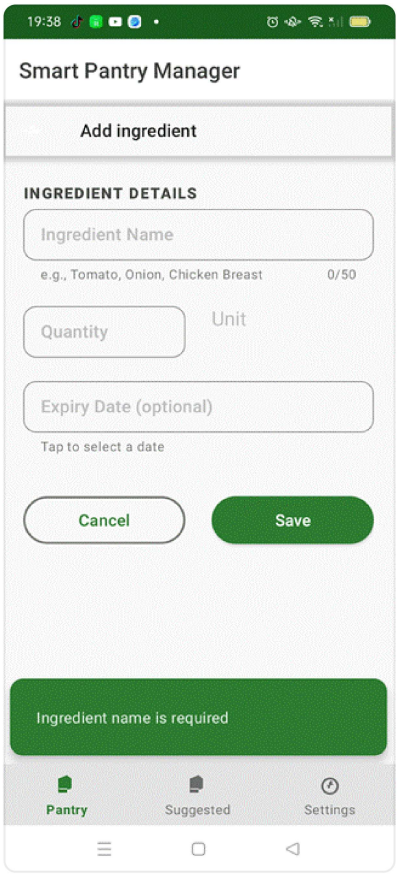


Figure 6: Validation error displayed when a required field is left blank.

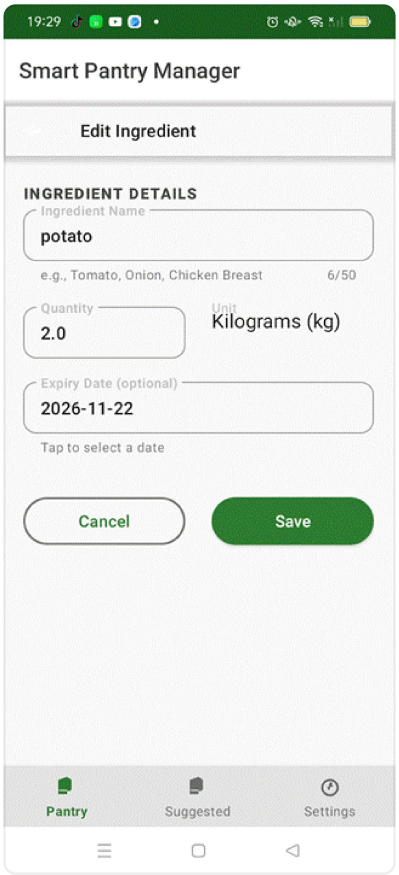


Figure 7: The edit screen with the existing pantry item's data pre-populated.

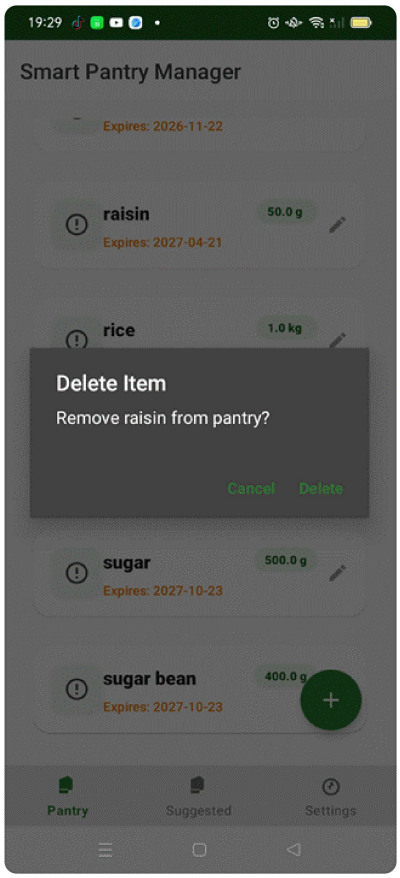


Figure 8: Delete confirmation dialogue triggered by a long-press on a pantry item.

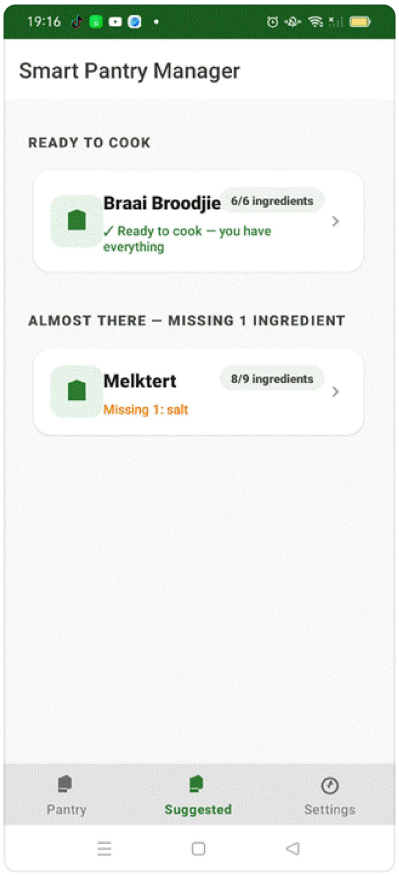


Figure 9: Suggested recipes with the strict 'Ready to Cook' list and the bonus 'Almost There' list.

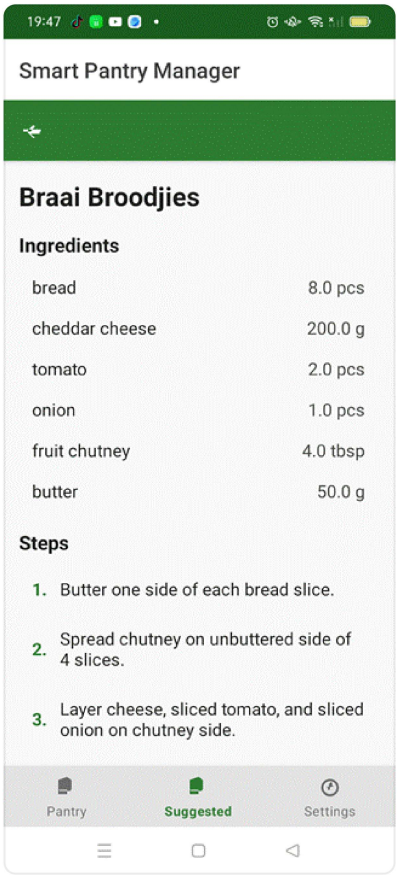


Figure 10: Recipe detail screen showing the required ingredients and the preparation steps.

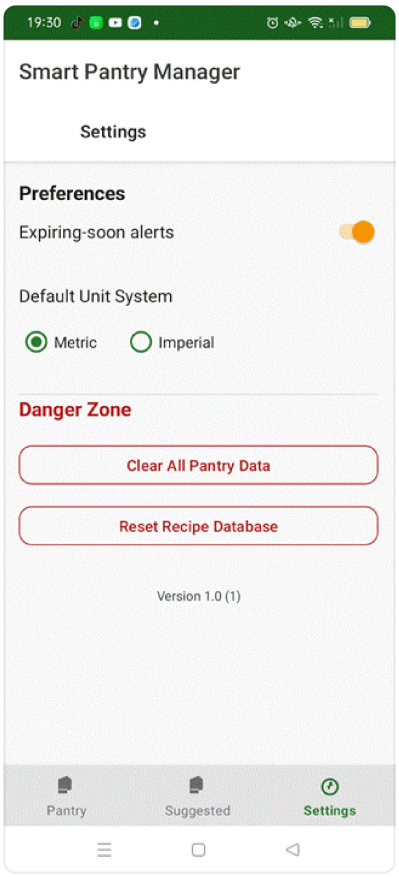


Figure 11: Settings screen.

4. Code Snippets

The following five snippets demonstrate the fundamental logic of the application.

4.1 Data Access Object (CRUD)

PantryItemDao.java

```
@Dao
public interface PantryItemDao {

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    long insert(PantryItem item);

    @Update
    int update(PantryItem item);

    @Delete
```

```
    int delete(PantryItem item);

    @Query("DELETE FROM pantry_items WHERE id = :itemId")
    int deleteById(long itemId);

    @Query("SELECT * FROM pantry_items ORDER BY ingredient_name ASC")
    LiveData<List<PantryItem>> getAllLive();

    @Query("SELECT * FROM pantry_items WHERE id = :itemId")
    LiveData<PantryItem> getByIdLive(long itemId);
}
```

4.2 Strict Ingredient Matching

IngredientMatcher.java

```
public static MatchResult checkRecipeMatch(Recipe recipe,
        List<RecipeIngredient> recipeIngredients,
        Map<String, PantryItem> pantryLookup) {

    List<String> missing = new ArrayList<>();

    for (RecipeIngredient ri: recipeIngredients) {
        String normName = normalizeName(ri.ingredientName);
        PantryItem pantryItem = pantryLookup.get(normName);

        if (pantryItem == null) {          // not in pantry at all
            missing.add(ri.ingredientName + " (not in pantry)");
            continue;
        }
        if (!areUnitsCompatible(pantryItem.unit, ri.unit)) {
            missing.add(ri.ingredientName + " (unit mismatch)");
            continue;
        }

        // Compare in base units (g, ml or pcs)
```

```
        double haveBase = convertToBaseUnit(pantryItem.quantity, pantryItem.unit);
        double needBase = convertToBaseUnit(ri.quantity, ri.unit);

        if (haveBase + 1e-9 < needBase) {  // epsilon for floating point
            missing.add(ri.ingredientName + " (quantity too low)");
        }
    }

    return new MatchResult(recipe.id, missing.isEmpty(),
            missing.size(), missing);
}
```

4.3 Reactive Suggestions with MediatorLiveData

SuggestedViewModel.java

```
private void setupMediatorLiveData() {
    LiveData<List<PantryItem>> pantryLive =
            pantryRepository.getAllPantryItems();
    LiveData<List<Recipe>> recipesLive =
            recipeRepository.getAllRecipes();

    // Recompute matches whenever the pantry OR the recipes change
    suggestedRecipes.addSource(pantryLive, pantryItems -> {
        List<Recipe> recipes = recipesLive.getValue();
        if (recipes != null) computeMatchesAsync(pantryItems, recipes);
    });
    suggestedRecipes.addSource(recipesLive, recipes -> {
        List<PantryItem> pantry = pantryLive.getValue();
        if (pantry != null) computeMatchesAsync(pantry, recipes);
    });
}

// Inside computeMatchesAsync(...) on a background thread:
if (mr.isStrictMatch) {
    // MAIN list: strictly matchable recipes only
```

```
        strictMatches.add(new RecipeWithMatch(r.id, r.name,
                totalIng, totalIng, true, 0, mr.missingIngredients));
    } else if (mr.missingCount == 1) {
        // Bonus list: missing exactly one ingredient
        almostMatches.add(new RecipeWithMatch(r.id, r.name,
                totalIng, totalIng - 1, false, 1, mr.missingIngredients));
    }
```

4.4 Date Picker (Crash Fix)

AddEditPantryFragment.java

```
private void showDatePicker() {
    // Guard against rapid double-taps showing the picker twice
    if (getChildFragmentManager()
            .findFragmentByTag(DATE_PICKER_TAG) != null) {
        return;
    }

    // Theming comes from the materialCalendarTheme attribute in the
    // app theme (a ThemeOverlay). Passing a full dialogue theme to
    // setTheme() here crashed layout inflation on some devices.
    MaterialDatePicker<Long> picker = MaterialDatePicker.Builder.datePicker()
            .setTitleText("Select Expiry Date")
            .setSelection(selectedExpiryDate != null
                    ? selectedExpiryDate
                    : MaterialDatePicker.todayInUtcMilliseconds())
            .build();

    picker.addOnPositiveButtonClickListener(selection -> {
        selectedExpiryDate = selection;
        SimpleDateFormat sdf =
                new SimpleDateFormat("yyyy-MM-dd", Locale.getDefault());
        binding.etExpiryDate.setText(sdf.format(new Date(selection)));
    });
```

```
    picker.show(getChildFragmentManager(), DATE_PICKER_TAG);
}
```

4.5 Efficient List Updates with DiffUtil

SuggestedAdapter.java

```
private static final DiffUtil.ItemCallback<ListItem> DIFF_CALLBACK =
        new DiffUtil.ItemCallback<ListItem>() {
    @Override
    public boolean areItemsTheSame(@NonNull ListItem oldItem,
                                   @NonNull ListItem newItem) {
        if (oldItem.getType() != newItem.getType()) return false;
        if (oldItem instanceof HeaderItem) {
            return ((HeaderItem) oldItem).headerType
                    == ((HeaderItem) newItem).headerType;
        }
        return ((RecipeItem) oldItem).recipe.recipeId
                == ((RecipeItem) newItem).recipe.recipeId;
    }

    @Override
    public boolean areContentsTheSame(@NonNull ListItem oldItem,
                                      @NonNull ListItem newItem) {
        RecipeWithMatch a = ((RecipeItem) oldItem).recipe;
        RecipeWithMatch b = ((RecipeItem) newItem).recipe;
        return a.name.equals(b.name)
                && a.matchedIngredients == b.matchedIngredients
                && a.missingCount == b.missingCount;
    }
};
```

5. Challenges and Solutions

5.1 Date Picker Crash on Theme Conflict

The most severe defect found during testing was a hard crash when tapping the expiry date field. The MaterialDatePicker was given a full MaterialComponents dialogue theme through Builder.setTheme(). On affected devices, the picker's internal layout inflated with that theme and threw an InflateException ("You must supply a layout_width attribute"), causing the app to crash. The fix was to stop passing a theme at build time and instead declare a proper ThemeOverlay (ThemeOverlay.MaterialComponents. MaterialCalendar-based) using the materialCalendarTheme attribute in the application theme. We also added a fragment-tag guard to prevent rapid double-taps from stacking two picker dialogues.

5.2 Strict Matching vs Usable Suggestions

The brief was that the main list of suggestions should contain only recipes that were strictly matchable. The initial builds had the problem of partial matches getting mixed in with the list which both breached the brief and led to very confusing results (a recipe missing 3 ingredients would look exactly the same as one missing none). The fix was to generate two separate lists in the ViewModel (strict matches and recipes missing exactly one ingredient) and to display them under distinct section headers with the appropriate colour coding to indicate the level of match. The "Almost There" list effectively turned the strict match requirement into a useful feature for the shopping list.

5.3 Normalising Names and Units

UUsers write pantry entries and recipe ingredients in many different ways, such as "Tomatoes" vs. “tomato,” "1 kg" vs. "1000 g,” and "2 tbsp" vs. "30 ml.” To account for these variations, IngredientMatcher carefully normalises ingredient names (lowercasing and trimming them, then singularising them with a mapping and simple suffix rules) and converts each quantity to a base unit—grams for mass, millilitres for volume, and pieces for count—before comparing them. Units from incompatible categories (for example, grams and millilitres) are treated as a mismatch rather than guessed.

5.4 Lifecycle Bugs Behind the Scenes

TTwo subtler bugs were also fixed during the hardening pass. The Add/Edit ViewModel observed LiveData with observeForever but didn’t remove the observer, which could cause the ViewModel to leak. The observer is now removed in onCleared(). In the original Suggested adapter, the row click listener wasn’t wired up correctly, so it never fired and recipe details couldn’t be reached. The adapter has now been rebuilt as a ListAdapter, with the listener connected to the ViewHolder.**6. Conclusion and Reflection**

The finished app meets the main goal: when your pantry is filled, it only shows recipes you can actually cook with what you have. Partial matches are kept out of the main list, and the "Almost There" section is a useful bonus. You can fully add, edit, and delete pantry items, with validation, a themed date picker, delete confirmation, and an undo option.

TThe biggest lesson from this project was that framework theming is about more than appearance. Applying the wrong theme to a component caused the whole app to crash, and resolving the issue meant consulting the Material Components source code rather than guessing. The project also highlighted the benefits of the MVVM pattern: once pantry items, recipes, and matches were set up as LiveData, features like the two-section suggestion list and automatic updates after edits became much easier to implement. If I were to extend the project, I’d add expiry notifications for items nearing their expiration date, along with a barcode scanner to make adding pantry items easier. id Developers (2024) Room persistence library. Available at: https://developer.android.com/training/data-storage/room (Accessed: 23 October 2026).Android Developers (2024) LiveData overview. Available at: https://developer.android.com/topic/libraries/architecture/livedata (Accessed: 23 October 2026).

Android Developers (2024) RecyclerView. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: 23 October 2026).

Material Design (2024) Material Components for Android: Date picker. Available at: https://github.com/material-components/material-components-android (Accessed: 23 October 2026).

GitHub (2026) Smart Pantry Manager – source repository. Available at: https://github.com/Naughtyboy25/Smart-Pantry-Mobile (Accessed: 23 October 2026).